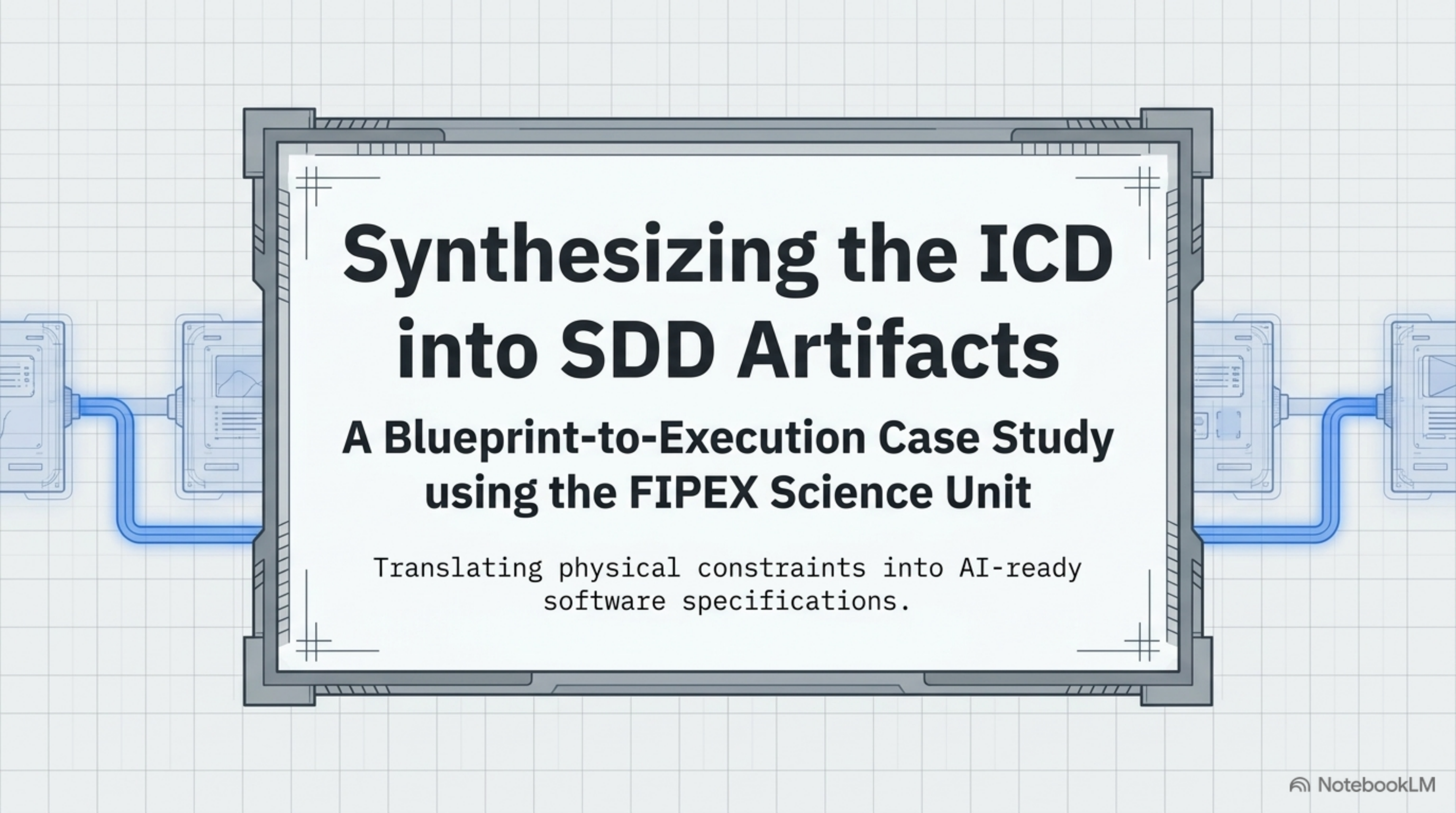




Synthesizing the ICD into SDD Artifacts

A Blueprint-to-Execution Case Study using the FIPEX Science Unit

Translating physical constraints into AI-ready
software specifications.

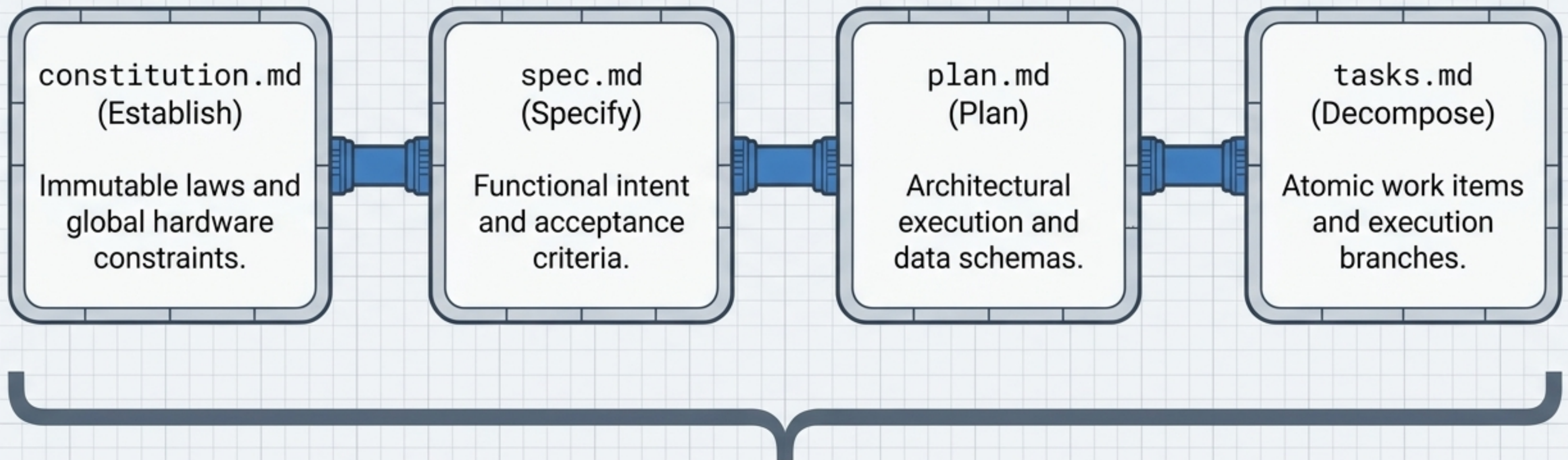
The ICD (The What)

- Physical boundaries
- Hardware realities
- Non-negotiable constraints
- Electrical/mechanical interfaces

The SDD (The How)

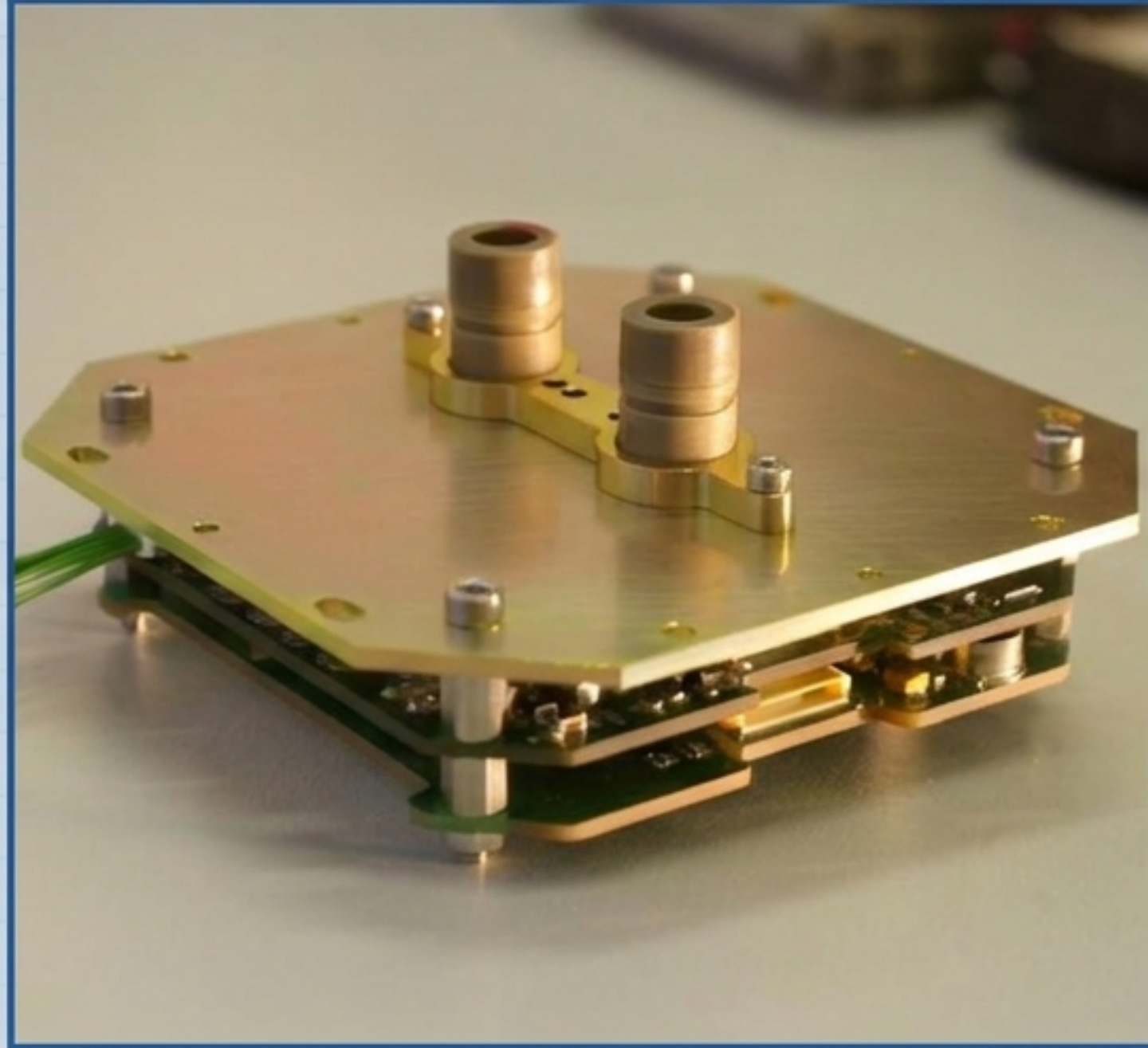
- Implementation logic
- Internal state management
- Data flow
- Error handling

Synthesis Insight: The bridge between these two documents is the **most critical point of failure** in systems engineering. If brittle, **'architectural drift'** occurs: the software assumes behaviors the hardware cannot support.



Each artifact narrows the context window, systematically eliminating ambiguity for human developers and AI agents.

Case Study Context: The FIPEX Science Unit



Mission:

Measures atomic oxygen in the lower thermosphere.

Hardware Reality:

Uses solid oxide electrolyte micro-sensors operating at extreme temperatures (600-700°C).

Software Implication:

Physics directly dictates software behavior. Power budgets, electrolysis laws, and thermal limits must become foundational software logic.

INPUT

2.8 Science Unit Serial Interface

Req: FPX-E-0210

The 3V3 I/O standard UART serial interface shall be used to control the SU from the OBC.
The serial I/F settings to use shall be:

...
...

Req: FPX-SW-0015

Data is transmitted in LITTLE ENDIAN BYTE order, i.e. LS byte first.

LS Bit of every byte is sent first, then the MS byte.

Req: FPX-SW-0015

Data is transmitted in LITTLE
ENDIAN BYTE order...

OUTPUT

Architectural Baselines

Global Data Serialization

****Rule:**** All data transmission must use LITTLE ENDIAN byte order (FPX-SW-0015).

****Implementation Example (C++):****

// Enforce Little Endian via compiler flags or explicit struct packing/conversion.

// Example:

#pragma pack(push, 1)

struct FIPEX_Command {

uint8_t command_id;

uint16_t payload_le; // Explicitly defined as Little Endian

};

#pragma pack(pop)

// Add static_assert to verify platform endianness if necessary.

// static_assert(__BYTE_ORDER__ == __ORDER_LITTLE_ENDIAN__, "Must compile on
Little Endian system");

Security by Construction: If this is not anchored globally, a stateless AI agent might default to Big Endian, entirely destroying the command parser.

INPUT

3. Science Unit Command and Control Interface

3.1 OBC Requirements for SU commanding

The OBC requirements is mainensrp the ercaaive length for for SU commanding:

Req: FPX-SW-0090

The SU will be commanded by uploading a byte encoded script. The length of every command can be either 5 or 7 byte, and the OBC shall determine the length, a command demans that the OKO commanded bry uploading a byte encoded script. The length of every command can be either 5 or 7 byte, and the OBC shall determine every command can be either 5 orr 7 byte, and the OBC shall determine the length, using the LEN field, and usising the LEN field.

Req: FPX-SW-0090

The SU will be commanded by uploading a byte encoded script... The length of every command can be either 5 or 7 byte, and the OBC shall determine the length... using the LEN field...

OUTPUT

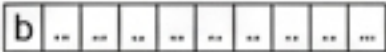
spec.md

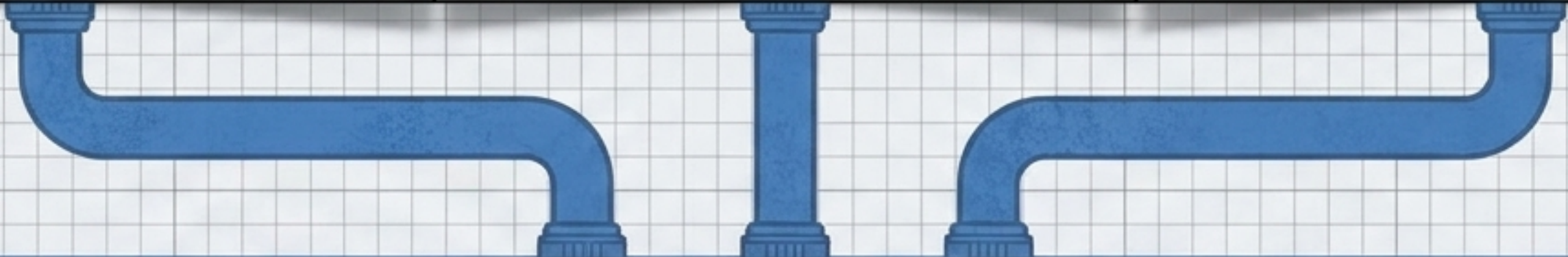
Acceptance Criteria

The software parser MUST dynamically determine command length (5 or 7 bytes) reading Byte 0 (LEN field).

Notice how physical byte specifications become verifiable, binary conditions for success.

ICD INPUT

Table 3-3: SU Command Packet Structure		
Start Byte (0x7E)	Command ID	XOR Checksum
		



plan.md

Communication Patterns

- **Protocol Choice:** UART Serial Interface (Req: FPX-E-0210)
- **Data Flow Schema:** Explicit XOR bitwise validation required prior to command execution.

ICD Input

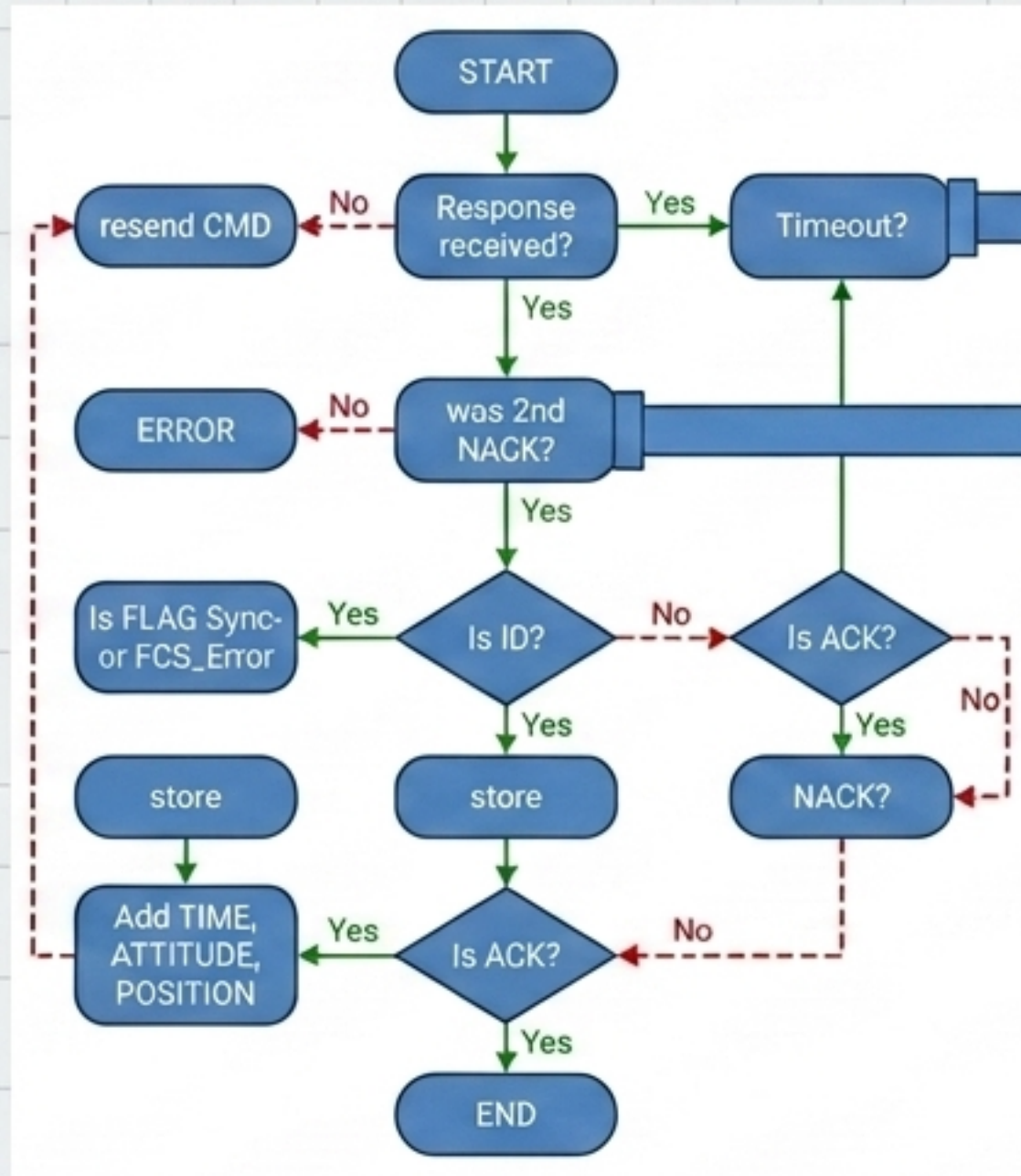


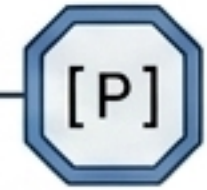
Figure 3-2: OBC Science Unit Command and Response Handling

tasks.md

Task 1:
Implement 500ms timeout
[FPX-SW-0270]



Task 2:
Implement XOR fail
retransmission logic





The Requirement:

FPX-SW-0300: 'OBC shall remove trailing zeroes after the XOR field'.

The Danger:

A human developer or AI might remove this formatting logic during a refactor to 'simplify' the code, unaware it breaks a physical constraint.

The SDD Fix (Explicit Rationale):

The specification must explicitly document the physical 'Why':

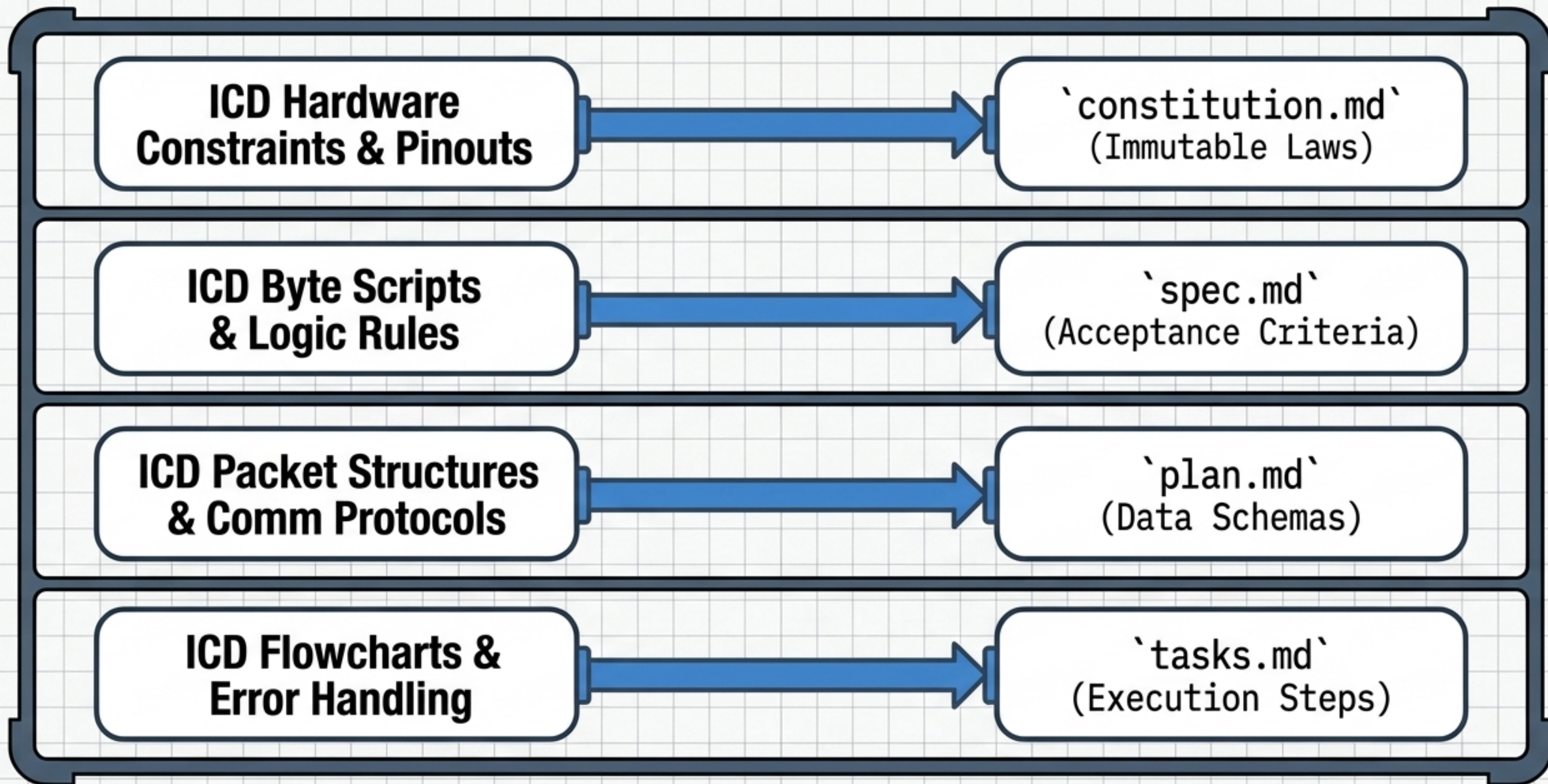
- Minimizing telemetry bandwidth during high-rate science orbits.
- Explicit decisions prevent architectural drift and fatal rework loops.

HARDWARE POWER CONSTRAINT TO SOFTWARE STATE MAPPING

State	Software Logic	Physical Power Constraint
Standby	Minimal logic, waiting	5 mA (+5V)
Science	High-rate acquisition, heater active	1900 mW peak
Error	Hardware fault, abort scripts	50 mW



Takeaway: Physical power budgets define the boundaries of the software state machine. The SDD must enforce these states to prevent hardware damage (e.g., blowing the 3000mW peak current limit).



The Spec is the source of truth. The code is merely a generated byproduct.